

GUIA TÉCNICO DO PROJETO

Video Editor

Ponto B

Next.js · Remotion · React · TypeScript · Zod

Documento técnico completo: arquitetura, conceitos, fluxo de dados, código comentado e referência rápida. Versão 2.0 — atualizada com SFX, paridade preview/render e suite de testes Vitest.

Stack	Next.js 14 (App Router) + Remotion 4 + TypeScript + Zod
Resolução	1080×1920 px (9:16 vertical), 30 fps
Cenas	11 tipos — Hook, FraseImpacto, ComparativoNumerico, VideoCitacao, ListaPontos, MiniCaso, TransicaoTexto, CTA, ConviteEvento, GraficoLinha, GraficoBarra
Testes	44 testes Vitest de paridade (preview vs. render)
Versão doc	2.0 — Maio 2026

Parte 1 — Next.js: o que é e o que usamos

1.1 React puro vs Next.js — a diferença central

Com React puro (Vite/SPA) o browser recebe um HTML quase vazio — o JavaScript roda no cliente e monta tudo. Next.js é um framework sobre React que adiciona um servidor Node.js, roteamento baseado em arquivos, capacidade de rodar código no servidor e geração de páginas estáticas ou híbridas. A diferença prática mais importante:

- **React puro (Vite):** todo código roda no browser. Sem servidor, sem API embutida.
- **Next.js:** alguns componentes rodam no servidor, outros no cliente. Você pode ter rotas de API dentro do mesmo projeto, no mesmo repositório.

1.2 App Router — o sistema de pastas como rotas

No Next.js 14 (que usamos), o roteamento funciona pela estrutura de pastas dentro de `src/app/`. Cada pasta vira uma rota, cada arquivo `page.tsx` dentro dela é a página daquela rota.

```
apps/web/src/app/  
page.tsx → rota: / (home – upload + editor)  
layout.tsx → layout raiz (envolve todas as páginas)  
jobs/  
page.tsx → rota: /jobs  
[jobId]/  
page.tsx → rota: /jobs/abc123 (detalhe de um job)  
api/  
jobs/  
route.ts → POST /api/jobs GET /api/jobs  
[jobId]/  
route.ts → GET /api/jobs/:id  
render/  
route.ts → POST /api/jobs/:id/render (SSE)  
especialistas/  
route.ts → GET /api/especialistas  
[slug]/  
logos/route.ts → GET /api/especialistas/:slug/logos
```

1.3 Route Handlers — as APIs dentro do Next.js

Os arquivos `route.ts` dentro de `api/` são equivalentes a Controllers no .NET. Você exporta funções com o nome do método HTTP: GET, POST, PUT, DELETE.

```
// apps/web/src/app/api/jobs/route.ts  
import { NextRequest, NextResponse } from 'next/server';
```

```
export async function POST(req: NextRequest) {

  const form = await req.formData();

  const videoFile = form.get('video') as File;

  const brief = form.get('brief') as string;

  const slug = form.get('especialista_slug') as string;

  // ... transcrição + análise Claude ...

  return NextResponse.json({ id: jobId, status: 'ready', scenes: reelProps });

}

export async function GET() {

  return NextResponse.json([/* lista de jobs */]);

}
```

O equivalente .NET seria um ApiController com [HttpPost] e [HttpGet]. A diferença é que aqui o arquivo já define a rota pela sua localização.

1.4 'use client' vs Server Components

Por padrão no Next.js 14, todo componente é Server Component — roda no servidor, sem acesso a browser APIs. Para usar interatividade, você precisa declarar `'use client'` no topo do arquivo.

- **Server Component (padrão):** roda no servidor. Pode fazer fetch de dados, ler arquivos. Não pode usar useState, hooks de efeito ou event handlers.
- **Client Component ('use client'):** roda no browser. Tem acesso a toda a API do React — hooks, eventos, DOM. É o que você usa em todas as páginas interativas do projeto.

```
// apps/web/src/app/page.tsx

'use client'; // necessário porque usa useState, useEffect, eventos

import { useState, useRef, useCallback, useEffect } from 'react';

export default function Home() {

  const [screen, setScreen] = useState<Screen>('upload');

  const [file, setFile] = useState<File | null>(null);

  const [brief, setBrief] = useState('');

  // ...

}
```

1.5 O layout.tsx — wrapper global

O arquivo `layout.tsx` na raiz de `src/app/` envolve todas as páginas. É onde ficam metadados globais, fontes e providers. O `{children}` é onde a página atual é injetada — como um `_Layout.cshtml` no Razor Pages.

```
// apps/web/src/app/layout.tsx

export const metadata: Metadata = {

  title: 'Ponto B — Editor de Reels',
```

```

};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang='pt-BR'>
      <body>{children}</body>
    </html>
  );
}

```

1.6 next.config.js — configurações do build

O `next.config.js` configura o webpack do Next.js. As duas coisas importantes: `transpilePackages` (compila pacotes locais como TypeScript) e o alias webpack que resolve `@pontob/schema` para o caminho físico no monorepo, sem precisar publicar o pacote no npm.

```

const nextConfig = {
  transpilePackages: ['@pontob/schema', '@remotion/player', 'remotion'],
  webpack(config) {
    config.resolve.alias = {
      ...config.resolve.alias,
      '@pontob/schema': path.resolve(__dirname, '../..packages/schema'),
    };
    return config;
  },
};

```

1.7 dynamic() — carregamento sem SSR

Alguns componentes usam APIs do browser que não existem no servidor (como o player de vídeo do Remotion). Para esses, usamos `dynamic()` do Next.js com `ssr: false`:

```

// apps/web/src/components/EditorView.tsx

import dynamic from 'next/dynamic';

// ReelPlayer usa <video> e APIs do browser — não pode rodar no servidor

const ReelPlayer = dynamic(
  () => import('./ReelPlayer').then((m) => m.ReelPlayer),
  { ssr: false, loading: () => <div>Carregando player...</div> }
);

```

Parte 2 — Remotion: React que vira vídeo

2.1 O modelo mental

Remotion funciona assim: você escreve um componente React normal. Em vez de renderizar no browser para um usuário, o Remotion renderiza o componente para cada frame do vídeo — como tirar um screenshot por frame — e depois junta tudo num .mp4.

- **Frame rate:** 30 fps = 30 renderizações por segundo. Um reel de 60s = 1.800 frames.
- **Resolução:** 1080×1920 pixels (9:16 — formato de Stories/Reels).
- **Animações:** não usam CSS animation. Usam matemática pura com o frame atual como variável.

2.2 Os hooks principais do Remotion

Dois hooks são a base de tudo. `useCurrentFrame()` retorna o frame atual (0, 1, 2...). `useVideoConfig()` retorna fps, largura, altura e duração total. Dividir frame por fps dá os segundos decorridos desde o início da cena.

```
import { useCurrentFrame, useVideoConfig } from 'remotion';

const MeuComponente = () => {
  const frame = useCurrentFrame(); // 0, 1, 2, 3...
  const { fps, width, height } = useVideoConfig();
  const segundos = frame / fps; // tempo decorrido
  const opacity = Math.min(1, frame / 30); // 0→1 em 1 segundo
  return <div style={{ opacity }}>Olá</div>;
};
```

2.3 spring() e interpolate() — animações fluidas

`spring()` simula física de mola — retorna um valor que vai de 0 a 1 com amortecimento natural. `interpolate()` mapeia esse valor de um range para outro. Esse padrão é usado em todas as cenas do projeto:

```
import { spring, interpolate, useCurrentFrame, useVideoConfig } from 'remotion';

const entrada = spring({
  frame,
  fps,
  config: { damping: 14, stiffness: 80 },
  // damping: amortecimento (maior = menos oscilação)
  // stiffness: rigidez (maior = mais rápido)
});

// entrada vai 0→1, opacity vai 0→1
const opacity = interpolate(entrada, [0, 1], [0, 1]);

// entrada vai 0→1, translateY vai 40→0 (sobe 40px)
```

```
const translateY = interpolate(entrada, [0, 1], [40, 0]);

return <div style={{ opacity, transform: `translateY(${translateY}px)` }}>...</div>;
```

2.4 AbsoluteFill e Sequence — estrutura das composições

AbsoluteFill é um div que ocupa 100% da composição (position: absolute, top/left/right/bottom: 0). Toda cena é construída empilhando AbsoluteFills. **Sequence** define quando uma cena começa (**from**) e quanto dura (**durationInFrames**). Dentro de um Sequence, **useCurrentFrame()** reseta para 0 — cada cena começa sempre do frame 0.

```
// Como o Reel.tsx organiza as cenas

<AbsoluteFill style={{ backgroundColor: '#0A1628' }}>

  /* Vídeo rodando por baixo, contínuo */
  <OffthreadVideo src={videoPath} />

  {sequencias.map(({ cena, inicioFrames, duracaoFrames }) => (
    <Sequence from={inicioFrames} durationInFrames={duracaoFrames}>
      <SceneRouter cena={cena} ... />
    </Sequence>
  ))}

</AbsoluteFill>

// O cursor acumula segundos, converte para frames só na hora de montar
let cursor = 0;

return props.cenas.map((cena) => {

  const duracaoFrames = Math.max(1, Math.round(cena.duracao_segundos * FPS));
  const inicioFrames = Math.round(cursor * FPS);
  cursor += cena.duracao_segundos; // acumula em segundos (evita arredondamento)

  return { cena, inicioFrames, duracaoFrames };

});
```

2.5 OffthreadVideo vs Video — a diferença crítica

Esse é o motivo de existirem dois arquivos que fazem a mesma coisa visual:

- **OffthreadVideo** (Reel.tsx — render servidor): renderiza o vídeo frame a frame de forma síncrona no processo Node.js. Extrai cada frame com precisão. Necessário para o render final.
- **Video** (ReelComposition.tsx — preview browser): usa o elemento <video> nativo. Funciona bem para preview interativo, mas não é adequado para render frame-a-frame pois é assíncrono.

```
// apps/remotion/src/compositions/Reel.tsx (render final)

<OffthreadVideo src={videoPath} startFrom={videoStartFrom}

style={{ width: '100%', height: '100%', objectFit: 'cover' }} />
```

```
// apps/web/src/components/ReelComposition.tsx (preview browser)

<Video src={videoUrl} startFrom={videoStartFrom}

style={{ width: '100%', height: '100%', objectFit: 'cover' }} />
```

2.6 delayRender / continueRender — esperar assets externos

O Remotion renderiza frames de forma síncrona. Se precisar carregar uma fonte antes de renderizar, você pausa o Remotion com **delayRender** e libera com **continueRender**. O FontLoader do projeto usa isso:

```
// apps/remotion/src/compositions/Reel.tsx

const FontLoader: React.FC<{ fonteUrl: string }> = ({ fonteUrl }) => {

const handleRef = useRef<number | null>(null);

useEffect(() => {

// Pausa o Remotion — ele não renderiza frames até continueRender

handleRef.current = delayRender(`loading-font-${fonteUrl}`);

const link = document.createElement('link');

link.rel = 'stylesheet'; link.href = fonteUrl;

link.onload = () => continueRender(handleRef.current!); // OK

link.onerror = () => continueRender(handleRef.current!); // erro: libera mesmo assim

document.head.appendChild(link);

}, [fonteUrl]);

return null; // não renderiza nada visualmente

};
```

2.7 Como o render é disparado pelo Next.js

Quando o usuário clica em Renderizar, o front-end chama **POST /api/jobs/:id/render**. Esse route handler executa o Remotion CLI via **spawn()** e transmite o progresso via Server-Sent Events (SSE). O Remotion emite linhas como **"Rendered 450/1590"** que são parseadas e encaminhadas para o browser:

```
// apps/web/src/app/api/jobs/[jobId]/render/route.ts (simplificado)

const stream = new ReadableStream({

start(controller) {

function send(data: object) {

controller.enqueue(encoder.encode(`data: ${JSON.stringify(data)}\n\n`));

}

const child = spawn(remotionBin, [

'render', 'Reel',

'--props', propsPath, // JSON das cenas editadas

'--output', outputPath,

], { cwd: REMOTION_DIR });
```

```
child.stderr.on('data', (chunk) => {

  const line = stripAnsi(chunk.toString());

  const m = FRAME_RE.exec(line); // 'Rendered 450/1590'

  if (m) send({ type: 'progress', frames: +m[1], total: +m[2] });

});

child.on('close', (code) => {

  if (code === 0) send({ type: 'done', outputPath });

  else send({ type: 'error', message: 'Remotion falhou' });

  controller.close();

});

}

});

return new Response(stream, { headers: { 'Content-Type': 'text/event-stream' } });
```

O front-end lê esse stream com a Streams API do browser, linha a linha, e atualiza a barra de progresso em tempo real (ver seção 3.3 — `handleRender`).

Parte 3 — Páginas e componentes React

3.1 page.tsx — a home (upload e seleção de especialista)

Essa é a primeira página que o usuário vê. Ela gerencia três estados: **upload**, **processing** e **editor**, e troca de tela conforme o pipeline avança.

```
type Screen = 'upload' | 'processing' | 'editor';

export default function Home() {
  const [screen, setScreen] = useState<Screen>('upload');
  const [job, setJob] = useState<Job | null>(null);
  const [file, setFile] = useState<File | null>(null);
  const [brief, setBrief] = useState('');
  const [especialistaSlug, setEspec] = useState('generico');
  const [especialistas, setEspecs] = useState<EspecialistaItem[]>([]);
  const [dragging, setDragging] = useState(false);
  const fileRef = useRef<HTMLInputElement>(null);

  // Carrega lista de especialistas ao montar
  useEffect(() => {
    fetch('/api/especialistas').then(r => r.json()).then(setEspecialistas);
  }, []);

  // Navegação entre telas baseada no estado
  if (screen === 'processing') return <ProcessingView fileName={file?.name ?? ''} />;
  if (screen === 'editor' && job) return <EditorView job={job} onNew={handleNew} />;
}
```

O `handleSubmit` monta um `FormData` com vídeo + brief + `especialista_slug` e faz `POST /api/jobs`. Na resposta bem-sucedida, salva o job no estado e navega para o editor. Erros reverts para a tela de upload.

```
async function handleSubmit() {
  const formData = new FormData();
  formData.append('video', file);
  formData.append('brief', brief);
  formData.append('especialista_slug', especialistaSlug);
  setScreen('processing');

  const res = await fetch('/api/jobs', { method: 'POST', body: formData });
  const data = await res.json();

  if (!res.ok) throw new Error(data.error);

  setJob(data as Job);
  setScreen('editor');
```

```
}

```

O `handleDrop` usa `useCallback` para que a referência da função não mude a cada render — importante quando a função é passada como prop para filhos que poderiam re-renderizar desnecessariamente com referência nova:

```
const handleDrop = useCallback((e: React.DragEvent) => {
  e.preventDefault();
  setDragging(false);
  const f = e.dataTransfer.files[0];
  if (f) handleFile(f);
}, []); // sem dependências – nunca recria

```

3.2 ProcessingView — feedback visual durante o pipeline

Componente de loading que mostra os 3 passos do pipeline com animação de estado. Não faz polling — apenas anima visualmente com `setTimeout` para dar feedback enquanto o `POST /api/jobs` processa no servidor.

```
const STEPS = [
  { key: 'transcribing', label: 'Transcrição', detail: 'Whisper extrai a fala...' },
  { key: 'analyzing', label: 'Análise', detail: 'Claude monta as cenas...' },
  { key: 'ready', label: 'Pronto', detail: 'Abrindo editor...' },
];

const [step, setStep] = useState(0);
useEffect(() => {
  const t1 = setTimeout(() => setStep(1), 1400);
  const t2 = setTimeout(() => setStep(2), 3200);
  return () => { clearTimeout(t1); clearTimeout(t2); }; // cleanup
}, []);

```

3.3 EditorView — o coração da aplicação

O componente mais complexo do projeto. Tem três colunas: lista de cenas, player de preview e painel de edição da cena selecionada. Gerencia o estado de todas as cenas, coordena render e refine.

Estado principal

```
const [scenes, setScenes] = useState<Cena[]>(job.scenes?.cenas ?? []);
const [selectedIdx, setSelectedIdx] = useState<number | null>(0);
const [rendering, setRendering] = useState(false);
const [renderProgress, setProgress] = useState<RenderProgress | null>(null);
const [outputPath, setOutputPath] = useState<string | null>(null);
const [renderError, setRenderError] = useState<string | null>(null);
const [refining, setRefining] = useState(false);
const [refineToast, setRefineToast] = useState<string | null>(null);

```

```
const previousOutput = job.outputPath; // render anterior da sessão
```

useMemo — cálculos derivados do estado

`useMemo` memoriza o resultado de um cálculo. Só recalcula quando as dependências mudam. No `EditorView` há dois usos críticos:

```
// Frame inicial do player quando o usuário seleciona uma cena
const initialFrame = useMemo(() => {
  if (selectedIdx === null) return 0;

  let acc = 0;

  for (let i = 0; i < selectedIdx; i++) {
    acc += scenes[i]?.duracao_segundos ?? 0; // soma duração das cenas anteriores
  }

  return Math.round(acc * FPS); // converte segundos → frames
}, [selectedIdx, scenes]);

// Props para o player – substitui caminhos de arquivo por URLs HTTP
const reelProps: ReelProps = useMemo(() => {
  const videoUrl = `/api/jobs/${job.id}/video`;
  const cenasComUrl = scenes.map((c) => {
    const cc = c as Record<string, unknown>;
    if ('video_path' in cc) return { ...c, video_path: videoUrl };
    return c;
  });
  return {
    duracao_total_estimada: totalSec,
    cenas: cenasComUrl,
    cor_primaria: job.scenes?.cor_primaria,
    cor_secundaria: job.scenes?.cor_secundaria,
    fonte_url: job.scenes?.fonte_url,
    fonte_familia: job.scenes?.fonte_familia,
  };
}, [scenes, totalSec, job.id]);
```

handleRender — Server-Sent Events

O `EditorView` lê o stream SSE do render linha a linha, acumulando chunks num buffer para lidar com eventos que chegam particionados:

```
async function handleRender() {

  // 1. Salva as cenas editadas antes de renderizar

  await fetch(`/api/jobs/${job.id}/scenes`, {
    method: 'PUT',
```

```

body: JSON.stringify({ ...job.scenes, cenas: scenes })),
});

// 2. Dispara o render – resposta é stream SSE

const res = await fetch(`/api/jobs/${job.id}/render`, { method: 'POST' });
const reader = res.body!.getReader();

let buffer = '';

while (true) {
  const { done, value } = await reader.read();
  if (done) break;

  buffer += decoder.decode(value, { stream: true });

  const events = buffer.split('\n\n'); // SSE separa eventos por linha dupla
  buffer = events.pop() ?? ''; // último pode estar incompleto

  for (const event of events) {
    const msg = JSON.parse(event.replace(/^data:\s*/m, '').trim());

    if (msg.type === 'progress') setRenderProgress(msg);
    if (msg.type === 'done') { setOutputPath(msg.outputPath); return; }
    if (msg.type === 'error') throw new Error(msg.message);
  }
}
}
}

```

Mutação de arrays de estado — o padrão imutável

```

// NUNCA mute diretamente – React não detecta mudanças em objetos mutados
// scenes.push(novaScene); // ← ERRADO

// Editar uma cena
onChange={() => {
  const next = [...scenes]; // copia o array
  next[selectedIdx!] = updated; // substitui o elemento
  setScenes(next); // React detecta nova referência e re-renderiza
}}

// Deletar
onDelete={() => {
  const next = scenes.filter((_, i) => i !== selectedIdx);
  setScenes(next);

  setSelectedIdx(Math.min(selectedIdx!, next.length - 1));
}}

// Mover para cima – swap de dois elementos

```

```

onMoveUp={() => {

  const next = [...scenes];

  [next[selectedIdx! - 1], next[selectedIdx!]] = [next[selectedIdx!], next[selectedIdx! - 1]];

  setScenes(next);

  setSelectedIdx(selectedIdx! - 1);

}}

```

Renderização condicional de telas

```

// Antes do JSX principal – substituições de tela inteira

if (rendering) return <RenderingScreen progress={renderProgress} />;

if (refining) return <RefiningScreen />;

if (outputPath) return <SuccessScreen jobId={job.id} outputPath={outputPath} />;

// Layout 3 colunas – só aparece sem estado especial

return (

  <main>

    <div>{/* Coluna 1: lista de cenas */}</div>

    <div><ReelPlayer props={reelProps} initialFrame={initialFrame} /></div>

    <div>{selected ? <SceneDetail ... /> : <EmptyState />}</div>

  </main>

);

```

3.4 SceneDetail — editor de campo por cena

Componente interno do EditorView que renderiza os campos de edição para a cena selecionada. Usa helpers locais para não repetir o padrão campo+label+input.

O helper textField

```
function textField(key: string, multiline = false) {
  if (!(key in c)) return null; // só renderiza se a cena tem esse campo

  const El = multiline ? 'textarea' : 'input';

  return (
    <div className={styles.field}>
      <label>{key}</label>
      <El
        value={String(c[key] ?? '')}
        onChange={(e) => onChange({ ...cena, [key]: e.target.value } as Cena)}
        // ↑ spread + override do campo alterado
        // ↑ [key] como computed property – TypeScript aceita
      />
    </div>
  );
}
```

O truque do `key in c`: como o schema define props diferentes por tipo de cena, `textField('titulo')` só aparece se a cena tem campo titulo. Isso evita condicionais explícitas para cada tipo.

O editor de palavras destacadas (versão atual — hex livre)

A versão anterior usava um `<select>` com enum de 3 opções (primaria/secundaria/branco). A versão atual usa um color picker HTML5 que salva hex diretamente — sem intermediação de um resolver:

```
// Suportado em: Hook, FraseImpacto, MiniCaso, CTA

const palavras = (Array.isArray(c['palavras_destacadas'])
  ? c['palavras_destacadas'] : []) as Array<{ palavra: string; cor: string }>;

{palavras.map((pw, i) => (
  <div key={i}>
    <input value={pw.palavra}
      onChange={(e) => {
        const next = [...palavras];
        next[i] = { ...next[i], palavra: e.target.value };
        onChange({ ...cena, palavras_destacadas: next } as Cena);
      }} />
    <input type='color' value={pw.cor}
      onChange={(e) => {
```

```
const next = [...palavras];

next[i] = { ...next[i], cor: e.target.value }; // hex direto

onChange({ ...cena, palavras_destacadas: next } as Cena);

}} />

</div>

))}
```

SFX Controls por cena

Cada cena expõe um painel de SFX com 5 botões de som (com preview de áudio), controle de volume inteiro 0-10 e campo de início em segundos:

```
const SFX_OPCOES = [

  { path: 'sfx/whoosh.mp3', label: 'whoosh', desc: 'entrada rápida' },
  { path: 'sfx/slide.mp3', label: 'slide', desc: 'movimento suave' },
  { path: 'sfx/pop.mp3', label: 'pop', desc: 'item aparecendo' },
  { path: 'sfx/ding.mp3', label: 'ding', desc: 'destaque / resultado' },
  { path: 'sfx/transition.mp3', label: 'transition', desc: 'passagem de cena' },

];

// Selecionar um som

onClick={() => onChange({ ...cena,

  sfx: ativo ? undefined : { path: op.path, volume: sfx?.volume ?? 1 }

} as Cena)}}

// Preview de áudio antes de selecionar (botão ■ por som)

function previewSfx(path: string, volume: number) {

  const audio = new Audio(`/api/sfx?file=${path}`);

  audio.volume = Math.min(1, volume / 10);

  audio.play();

}
```

3.5 ReelComposition.tsx — o preview fiel no browser

Espelha a lógica do `apps/remotion/src/compositions/Reel.tsx`, mas usa componentes do browser. Existe separado porque o `@remotion/player` não consegue usar `OffthreadVideo` (só funciona no processo de render do Node.js).

O tema inline

ReelComposition.tsx não pode importar de `apps/remotion` (bundle separado), então duplica localmente os design tokens. Uma suite de 44 testes Vitest garante que os valores permanecem idênticos entre os dois contextos:

```
const colors = { navy: '#0A1628', red: '#E63946', yellow: '#F4C430', ... };

const typography = { weightHero: 900, sizeTitle: 72, trackingTight: -1.5, ... };

const spacing = { xs: 8, sm: 16, md: 32, lg: 64, xl: 96, xxl: 144 };
```

buildTokenCorMap — destaques de múltiplas palavras

A versão anterior do preview comparava token por token (só palavras isoladas). A versão atual usa `buildTokenCorMap` que desliza uma janela sobre tokens não-whitespace, suportando expressões de múltiplas palavras:

```
// Texto splitado preservando espaços: 'COMENTE AQUI' → ['COMENTE', ' ', 'AQUI']
const tokens = cena.texto.split(/(\s+)/);

const corMap = buildTokenCorMap(tokens, cena.palavras_destacadas ?? []);

// Cada token vira um <span> com a cor do mapa
{tokens.map((token, i) => (
  <span key={i} style={{ color: corMap[i] ?? colors.white }}>
    {token}
  </span>
))}

// buildTokenCorMap: varre sequências consecutivas de não-whitespace
// e atribui a cor hex diretamente (sem resolver – campo cor é hex livre)
for (const pw of palavras) {
  const palavraTokens = pw.palavra.toLowerCase()
    .replace(/[,!?:;]/g, '').split(/(\s+)/).filter(Boolean);
  // desliza janela de N tokens sobre o array...
  if (match) candidates.forEach(idx => { corMap[idx] = pw.cor; });
}
```

makeCorDestaque — resolver de cor (legado, ainda presente)

Ainda usado no CtaScene para o campo `cor_seta`, que continua sendo um enum (primaria/secundaria/branco) em vez de hex livre:

```
function makeCorDestaque(corPrimaria?: string, corSecundaria?: string) {
  return (cor: 'primaria' | 'secundaria' | 'branco'): string => {
    if (cor === 'primaria') return corPrimaria ?? colors.red;
    if (cor === 'secundaria') return corSecundaria ?? colors.yellow;
    return colors.white;
  };
}

// Uso: const setaCor = resolver(cena.cor_seta ?? 'primaria');
```


Parte 4 — Schema, dados e o fluxo completo

4.1 Zod como fonte de verdade

O arquivo `packages/schema/scenes.ts` define todos os tipos com Zod. É importado por três partes distintas do projeto que precisam concordar sobre o shape dos dados:

```
packages/schema/scenes.ts ← fonte de verdade
↑ ↑ ↑
services/analysis apps/remotion apps/web
(Claude valida) (render usa) (editor e player usam)
```

O schema usa `discriminatedUnion` no campo `tipo`. O TypeScript estreita o tipo automaticamente em switches:

```
// Sem discriminatedUnion — TypeScript não sabe o tipo
function renderCena(cena: Cena) {
  cena.titulo; // ← ERRO: titulo não existe em todos os tipos
}

// Com discriminatedUnion — o switch estreita o tipo
switch (cena.tipo) {
  case 'Hook':
    cena.titulo; // ✓ TypeScript sabe que é Hook
    break;
  case 'ComparativoNumerico':
    cena.metrica_nome; // ✓ TypeScript sabe que é ComparativoNumerico
    break;
}
```

WordHighlightSchema — migração de enum para hex livre

O campo `cor` era um enum de 3 opções (primaria/secundaria/branco). Foi migrado para `z.string()` aceitando qualquer hex — isso eliminou a necessidade do resolver em todos os tipos que usam `palavras_destacadas`:

```
// Antes
cor: z.enum(['primaria', 'secundaria', 'branco'])

// Agora
cor: z.string().min(1) // hex livre, ex: '#E63946'
```

SfxSchema — novo no sistema

```
export const SfxSchema = z.object({
  path: z.string().min(1), // 'sfx/whoosh.mp3'
  volume: z.number().min(0).max(10).optional(), // 0-10
  inicio_segundos: z.number().min(0).optional(),
```

```

fim_segundos: z.number().min(0).optional(),
});

// Todos os 11 tipos de cena têm: sfx: SfxSchema.optional()

```

4.2 O fluxo completo de dados

```

1. UPLOAD (browser)

FormData(video + brief + slug) → POST /api/jobs

2. TRANSCRIÇÃO (Node.js → Python)

execFile(python, ['run.py', '--input', videoPath, '--output', transcriptPath])
→ jobs//transcript.json (Whisper large-v3)

3. ANÁLISE (Node.js → Anthropic API)

analyze({ transcript, especialista, brief })
→ ReelProps validado pelo schema Zod
→ jobs//scenes.json

4. RESPOSTA

{ id, status: 'ready', scenes: ReelProps }
→ EditorView recebe via setJob()

5. EDIÇÃO (browser)

useState<Cena[]> – edições mudam o array local
useMemo(reelProps) → ReelPlayer atualiza o preview

6. RENDER (Node.js → Remotion CLI)

PUT /api/jobs/:id/scenes (salva edições)
POST /api/jobs/:id/render (spawn remotion CLI)
SSE stream → barra de progresso no browser
→ jobs//out/reel.mp4

7. DOWNLOAD

GET /api/jobs/:id/download → arquivo .mp4
DELETE /api/jobs/:id/cleanup (opcional – limpa arquivos temporários)

```

4.3 analysis — integração com a API da Anthropic

A função `analyze()` em `services/analysis/src/claude.ts` envia a transcrição e o perfil do especialista para o Claude e valida a resposta com o schema Zod. Tem retry automático em caso de resposta inválida:

```

// services/analysis/src/claude.ts

const DEFAULT_MODEL = process.env.CLAUDE_MODEL ?? 'claude-sonnet-4-6';

```

```

export async function analyze(params: AnalyzeParams): Promise<AnalyzeResult> {
  // Tenta até 3 vezes em caso de JSON inválido ou falha de validação Zod
  for (let tentativa = 1; tentativa <= 3; tentativa++) {

    const msg = await client.messages.create({
      model: DEFAULT_MODEL,
      max_tokens: 4096,
      system: SYSTEM_PROMPT,
      messages: [{ role: 'user', content: buildUserPrompt(params) }],
    });

    const text = stripMarkdownFence(msg.content[0].text);
    const parsed = ReelPropsSchema.parse(JSON.parse(text));

    return { scenes: parsed, metadata: { model: DEFAULT_MODEL, tentativas } };
  }
}

// analysis-bridge.ts — re-export para o Next.js resolver via tsconfig paths
export { analyze, refine } from '../../../services/analysis/src/claude';

```

4.4 lib/db.ts — armazenamento de especialistas

Os especialistas são armazenados como arquivos JSON na pasta **especialistas/** na raiz do repo. O **db.ts** é uma camada de abstração sobre leitura/escrita desses arquivos — não usa banco de dados real. O **generico.json** é o fallback do sistema.

```

// Cada especialista: especialistas/.json

export function getEspecialista(slug: string): EspecialistaRow | null {
  const fp = path.join(ESPECIALISTAS_DIR, `${slug}.json`);
  if (!fs.existsSync(fp)) return null;
  return JSON.parse(fs.readFileSync(fp, 'utf-8'));
}

// Fallback automático para generico.json

export function getEspecialistaOrGenerico(slug: string): EspecialistaRow {
  return getEspecialista(slug) ?? getEspecialista('generico') ?? DEFAULTS;
}

// EspecialistaRow inclui: slug, nome, cargo, cor_primaria, cor_secundaria,
// fonte_url, fonte_familia, publico_alvo, tom_de_voz, cta_formato,
// cta_palavra, vocabulario, palavras_proibidas, metricas

```

Parte 5 — Conceitos React usados no projeto

5.1 useState e re-render

Todo estado que muda a UI precisa estar no `useState`. Quando você chama `setScenes(next)`, o React compara a referência antiga com a nova — se for diferente, re-renderiza. Por isso nunca se faz `scenes.push()` direto (mutação) — sempre cria um novo array com `[...scenes]`.

5.2 useEffect e ciclo de vida

`useEffect` com array vazio `[]` roda uma vez na montagem. Com dependências `[x, y]` roda quando `x` ou `y` mudam. Com `cleanup` (`return () => {...}`) limpa antes do próximo efeito ou na desmontagem — como o `clearTimeout` no `ProcessingView`.

5.3 useMemo e performance

`useMemo` memoriza o resultado de um cálculo pesado. Só recalcula quando as dependências mudam. No `EditorView`, `reelProps` e `initialFrame` usam `useMemo` para não recalculer a cada render — o player não re-inicializa sem necessidade.

5.4 useRef

`useRef` não causa re-render quando muda. É usado para dois casos: guardar uma referência ao DOM (`fileRef` aponta para o input hidden de arquivo), ou guardar um valor mutável sem re-render (`handleRef` guarda o id do `delayRender` no `FontLoader`).

5.5 useCallback

Memoriza uma função entre renders. `handleDrop` usa `useCallback` para que a referência da função não mude a cada render — importante quando a função é passada como prop para filhos que teriam re-render desnecessário com referência nova.

5.6 TypeScript + React: padrões usados no projeto

```
// Tipo inferido do Zod – DRY total, sem duplicação
export type Cena = z.infer<typeof CenaSchema>;

// Extract<> – estreita um union para um membro específico
type HookCena = Extract<Cena, { tipo: 'Hook' }>;

// Type guard com predicado – compilador entende o tipo após a chamada
const hookCena = props.cenas.find(
  (c): c is Extract<Cena, { tipo: 'Hook' }> => c.tipo === 'Hook'
);

// Record<string, unknown> – escape hatch para acessar campo por nome dinâmico
const c = cena as Record<string, unknown>;

if ('titulo' in c) { /* ... */ }
```

```
// never no default do switch – TypeScript verifica exhaustiveness  
  
default: {  
  
  const _exhaustive: never = cena; // erro em compilação se tipo não coberto  
  
  return <PlaceholderScene />;  
  
}
```

Parte 6 — Paridade preview/render e testes automáticos

6.1 O problema — por que paridade é crítica

O preview usa `<Video>` (browser) e o render usa `<OffthreadVideo>` (Node.js). Isso exige que os overlays sejam implementados em dois lugares: `ReelComposition.tsx` e `apps/remotion/src/scenes/*.tsx`. Sem controle, os dois ficam fora de sincronia silenciosamente.

6.2 As 7 divergências corrigidas

Cena	Divergência encontrada	Correção aplicada
FraseImpacto	Gradiente iniciava em 30% no preview, 35% no render	Preview ajustado para 35%
ComparativoNumerico	Preview sem footer 'em {metrica_unidade}'	Footer adicionado ao preview
ComparativoNumerico	Preview com font-size fixo 120px (sem breakpoints)	5 níveis: 120/96/72/56/44px por n.º chars
VideoCitacao	cargo_mentor com weightTitle (800) em vez de 500	Corrigido para weightCaption + opacity 0.9
VideoCitacao	Stagger das frases: 12 frames no preview, 8 no render	Corrigido para i * 8 frames em ambos
ListaPontos	Título sem animação translateY no preview	tituloY spring adicionado ao preview
Hook	Gradiente inferior ausente no render (só no preview)	linear-gradient adicionado em HookScene.tsx

6.3 Suite de testes Vitest — 44 testes

Os testes ficam em `tests/parity/preview-render-parity.test.ts` e rodam com `npm run test:parity`. Importam `CenaSchema` diretamente — um novo tipo adicionado ao schema quebra automaticamente o assert de cobertura:

```
import { CenaSchema } from '@pontob/schema';

// Extrai os tipos do discriminatedUnion dinamicamente
const tiposSchema = (CenaSchema.options as Array<...>)
  .map((opt) => opt.shape.tipo.value);

// ['Hook', 'FraseImpacto', 'ComparativoNumerico', ...]

// Para cada tipo, verifica que existe overlay no preview E case no SceneRouter
for (const tipo of tiposSchema) {
  expect(PREVIEW).toContain(tipoParaOverlay[tipo]); // HookOverlay, etc.
  expect(routerSection).toContain(`case "${tipo}"`); // no SceneRouter
}
```

Grupo (describe)	Testes	O que verifica
Schema coverage	2	Todos os tipos têm overlay + case no SceneRouter

SFX — overlays	11	Cada overlay tem bloco <Audio> condicional a cena.sfx
SFX — defaults	11	migrarTipo usa o sfx correto por tipo de cena
GraficoLinha	4	W/H/PADs, formatVal prefixado, mostrar_area, SVG overflow
TransicaoTexto	2	Padding spacing.xml, gradiente em 40%
ComparativoNumerico	2	Footer metrica_unidade, 5 breakpoints de font-size
VideoCitacao	3	Stagger i*8, cargo weightCaption + opacity 0.9, trackingNormal
ListaPontos	2	tituloY com translateY, ITEM_DELAY = 12
Hook	1	linear-gradient ou GradienteInferior presente
FraseImpacto	1	Gradiente stop em 35%
Total	44	

Referência rápida — mapa do projeto

Arquivo / pasta	O que faz
packages/schema/scenes.ts	Schema Zod — fonte de verdade de todos os tipos de cena
apps/web/src/app/page.tsx	Home: upload de vídeo, seleção de especialista, transição de telas
apps/web/src/components/EditorView.tsx	Editor principal: lista de cenas, player, painel de edição, SFX UI
apps/web/src/components/ReelComposition.tsx	Preview do reel no browser (usa <Video>, não OffthreadVideo)
apps/web/src/components/ProcessingView.tsx	Tela de loading durante transcrição e análise
apps/web/src/app/api/jobs/route.ts	POST /api/jobs: transcrição (Whisper) + análise (Claude)
apps/web/src/app/api/jobs/[jobId]/render/route.ts	POST /api/jobs/:id/render: dispara Remotion, SSE de progresso
apps/web/src/lib/db.ts	Leitura/escrita de especialistas em JSON
apps/remotion/src/theme.ts	Design tokens: cores, tipografia, spacing, buildTokenCorMap
apps/remotion/src/compositions/Reel.tsx	Composição Remotion: OffthreadVideo, SceneRouter, FontLoader
apps/remotion/src/scenes/*.tsx	Uma Scene por tipo de cena — animações spring/interpolate + SFX
services/analysis/src/claude.ts	Integração Anthropic: analyze() e refine() com retry automático
services/analysis/src/prompt.ts	System prompt e user prompt enviados ao Claude
especialistas/.json	Dados de cada especialista (cores, fonte, CTA, vocabulário...)
jobs//	Pasta de cada job: vídeo, transcript.json, scenes.json, out/reel.mp4
tests/parity/preview-render-parity.test.ts	44 testes Vitest de paridade preview vs. render
vitest.config.ts	Config Vitest com alias @pontob/schema → packages/schema/scenes.ts